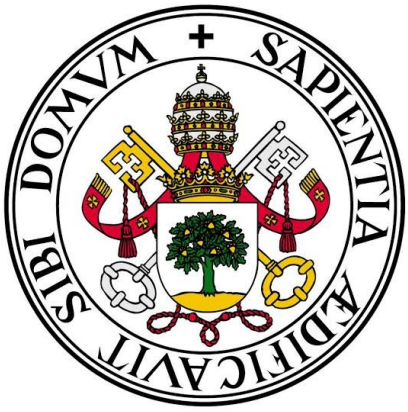
Primera entrega

SISTEMAS INTELIGENTES: BÚSQUEDA



Izan Arraez Pulido

Raúl Desiderio Moto Buambo

Ejercicio 1

Espacio de estados

Estados: Los estados son cada una de las casillas (B0 hasta B17)
Estado inicial: B0
Acciones: Desplazarse 1 o 2 casillas si estamos en una ficha blanca y 1 o 4 si estamos en una negra
Test Objetivo: Estar en B17
Coste del camino: El coste del camino es 1 (movimiento)

No informada

DFS

CURRENT PATH:
B0 --> N1 --> B2 --> B3 --> N5 --> B6 --> B7 --> B9 --> N10 --> B11 --> N13 --> N14 --> B15 --> B16 --> B17 (Goal)
Path to last Goal Node: B0 --> N1 --> B2 --> B3 --> N5 --> B6 --> B7 --> B9 --> N10 --> B11 --> N13 --> N14 --> B15 --> B16 --> B17 (Goal) Cost: 14.0
Nodes expanded: 15

En esta búsqueda da este resultado debido que ha estado cayendo por el lado izquierdo, chocando con nodos repetidos y tomando la siguiente ruta es decir el lado derecho repetidamente hasta toparse con la meta.

BFS

CURRENT PATH:
B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal)
Path to last Goal Node: B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal) Cost: 6.0
Nodes expanded: 92

En esta búsqueda da este resultado debido a que ha recorrido todos los nodos posibles y ha encontrado el mejor camino optimo.

Lower Cost First

CURRENT PATH:
B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal)
Path to last Goal Node: B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal) Cost: 6.0
Nodes expanded: 92

En esta búsqueda da este resultado debido a que ha seguido la ruta con menos valor, pero al tener todos el mismo el programa sigue su ejecución y da este resultado.

Informada

Heuristica: número de casillas restantes hasta el objetivo/número máximo de movimientos
Explicación: suponiendo que desde cualquier estado se puede avanzar siempre el máximo de 4 casillas por movimiento. De esta forma se estima el mínimo numero de movimientos necesarios para alcanzar el objetivo. Es admisible por que nunca se va a sobreestimar el coste.

A*

CURRENT PATH:
B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal)
Path to last Goal Node: B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal) Cost: 6.0
Nodes expanded: 23.

En el A* encuentra la ruta en: B0 --> N1 --> N5 --> B9 --> B11 --> N13 --> B17 (Goal), con un coste total de 6, ya que se realizan 6 movimientos y cada movimiento tiene un coste de 1.

Best First
CURRENT PATH:
B0 --> N1 --> N5 --> B9 --> N10 --> N14 --> B15 --> B17 (Goal)
Path to last Goal Node: B0 --> N1 --> N5 --> B9 --> N10 --> N14 --> B15 --> B17 (Goal) Cost: 7.0
Nodes expanded: 8
En Best First encuentra la ruta: B0 --> N1 --> N5 --> B9 --> N10 --> N14 --> B15 --> B17 (Goal), con un coste total de 7, es decir 7 movimientos. Este algoritmo selecciona siempre el nodo que parece estar mas cerca del objetivo según la heurística.
F(n) = h(n)

Ejercicio 2
Espacio de estados

Estados: Lista de tortitas donde muestran su orden de arriba a abajo
Estado inicial: [2,4,1,3]
Acciones: Voltear las n tortitas introducido la espátula, estando n entre 2 y el total de las tortitas (4)
Test Objetivo: Llegar a [1,2,3,4]
Coste del camino: El coste del camino es 1 (vuelta)

No informada

DFS

CURRENT PATH:
[2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [2,1,4,3] --> [1,2,4,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [4,3,1,2] --> [1,3,4,2] --> [3,1,4,2] --> [4,1,3,2] --> [1,4,3,2] --> [2,3,4,1] --> [3,2,4,1] --> [4,2,3,1] --> [2,4,3,1] --> [3,4,2,1] --> [4,3,2,1] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [2,1,4,3] --> [1,2,4,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [4,3,1,2] --> [1,3,4,2] --> [3,1,4,2] --> [4,1,3,2] --> [1,4,3,2] --> [2,3,4,1] --> [3,2,4,1] --> [4,2,3,1] --> [2,4,3,1] --> [3,4,2,1] --> [4,3,2,1] --> [1,2,3,4] (Goal) Cost: 19.0
Nodes expanded: 21

En esta búsqueda da este resultado debido que ha estado cayendo por el lado izquierdo, chocando con nodos repetidos y tomando la siguiente ruta es decir el lado derecho repetidamente hasta toparse con la meta.

BFS

CURRENT PATH:
[2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 26

En esta búsqueda da este resultado debido a que ha recorrido todos los nodos posibles y ha encontrado el mejor camino optimo.

Lower Cost First

CURRENT PATH:
[2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 27

En esta búsqueda da este resultado debido a que ha seguido la ruta con menos valor, pero al tener todos el mismo el programa sigue su ejecución y da este resultado.

Informada
Heurística 1: Coste de las vueltas según la posición de la tortita más grande

Esta heurística se basa en la posición de la tortita más grande, es decir si esta se encuentra en su posición vale 0, si se encuentra la primera vale 1 y si está en el medio vale 2. Estos números son el número de vueltas que habría que hacer para que estuviera en su sitio. Esta heurística es admisible ya que el número máximo de vueltas es 2 que es inferior o igual al coste real siendo este de 4.

A*

CURRENT PATH:
[2,4,1,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 15

En esta búsqueda da este resultado debido a que ha sumado el los pasos mas la heuristica dada de cada nodo, según el numero de vueltas de la ultima tortita. Este da un resultado mejor que las otras y mas bajo dado un resultado optimo (2->1->0->0->0).

Best First

CURRENT PATH:
[2,4,1,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [4,2,1,3] --> [3,1,2,4] --> [2,1,3,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 6

En esta búsqueda da este resultado debido a que solo ha tenido que expandir 6 nodos, porque se guía solo según las heuristicas y ha dado la casualidad de encontrar el camino de forma rápida

Heurística 2: Coste según la distancia por cercanía

Esta heurística se basa en la posición de la tortita según la cercanía al numero, es decir si tienes este bloque [2,4,1,3] como máximo puede dar 4 ya que no hay ninguno numero que este cerca del que debería estar,ni el ultimo numero esta tocando la superficie donde estén puestas las tortitas. Por lo que esta heurística es admisible ya que el numero máximo de vueltas es 4 que es inferior o igual al coste real siendo este de 4.

A*

CURRENT PATH:
[2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 9

En esta búsqueda da este resultado debido a que ha sumado el los pasos más la heuristica dada de cada nodo, según la cercanía de cada de cada tortita (1 esta cerca de 2, 2 esta cerca de 1 y 3, 3 esta cerca de 2 y 4 y 4 esta cerca de 3 y de la superficie). Este da un resultado mejor que las otras y más bajo dado un resultado optimo (4→3→2→1→0).

Al compara entre la heuristica 1 y la 2 (H1 expande 15 nodos y H2 expande 9 nodos), queda demostrado que la H2 es mas informada. Esto se debe a que los valores son mas precisos y cercanos al coste real, por lo que guía el algoritmo de forma mas directa a la meta.

Best First

CURRENT PATH:
[2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal)
Path to last Goal Node: [2,4,1,3] --> [1,4,2,3] --> [4,1,2,3] --> [3,2,1,4] --> [1,2,3,4] (Goal) Cost: 4.0
Nodes expanded: 5

En esta búsqueda da este resultado debido a que solo ha tenido que expandir 5 nodos, porque se guía solo según las heuristicas y ha dado la casualidad de encontrar el camino de forma rápida.